

Insurance AI Assistant

Auth Wiring — Frontend ↔ Backend

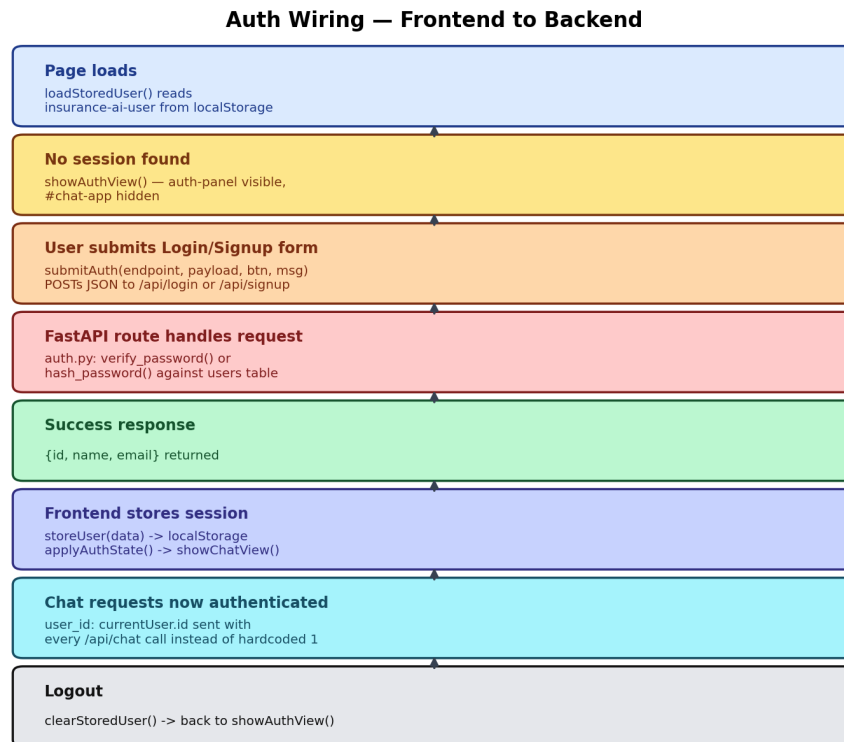
Login / Signup / Logout feature — how the two sides connect

This doc explains exactly how the new authentication feature is wired: the real backend endpoints and models, and how the frontend's session logic (as implemented in the v2 UI) calls them, stores the session, and gates the chat behind login.

1. What Changed, In One Line

Chat used to run as a fixed demo user (USER_ID = 1) with no login. Now every chat is tied to a real authenticated account: the frontend gates access behind a Login/Sign-up screen, the backend verifies credentials and issues a user record, and the frontend keeps that user's id in localStorage so every subsequent request is scoped to them.

2. End-to-End Flow



3. Backend — What Actually Runs

Three new files plus one column added to the existing users table. This is the exact code currently running.

3.1 Database: password_hash column

app/database/models.py — User class

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(120), nullable=False)
    email = Column(String(150), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    chats = relationship("ChatHistory", back_populates="user", cascade="all, delete-orphan")
```

- Added password_hash (String, 255 chars — enough for a bcrypt hash) to the existing users table
- Never store the plain password — only the bcrypt hash goes to MySQL

3.2 Password hashing service

app/services/auth_service.py

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)
```

- passlib's CryptContext with bcrypt — industry-standard one-way hashing
- hash_password() runs at signup, verify_password() runs at login — the plain password is never compared directly, only its hash

3.3 Request/response shapes

app/schemas/auth.py

```
from pydantic import BaseModel, EmailStr, Field

class SignupRequest(BaseModel):
    name: str = Field(..., min_length=1, max_length=120)
    email: EmailStr
    password: str = Field(..., min_length=6, max_length=128)

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class UserResponse(BaseModel):
    id: int
    name: str
    email: str

class Config:
    from_attributes = True
```

- SignupRequest / LoginRequest — Pydantic validates email format (EmailStr) and minimum password length before the request even reaches the handler
- UserResponse — deliberately excludes password_hash; this is the exact shape the frontend's storeUser(data) receives

3.4 The two endpoints

app/api/routes/auth.py

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database.session import get_db
from app.database.models import User
from app.schemas.auth import SignupRequest, LoginRequest, UserResponse
from app.services.auth_service import hash_password, verify_password

router = APIRouter()

@router.post("/signup", response_model=UserResponse)
def signup(req: SignupRequest, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == req.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="Email already registered")

    user = User(name=req.name, email=req.email, password_hash=hash_password(req.password))
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.post("/login", response_model=UserResponse)
def login(req: LoginRequest, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == req.email).first()
    if not user or not verify_password(req.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid email or password")
    return user

```

- POST /api/signup — checks email isn't already registered, hashes the password, creates the user, returns it
- POST /api/login — looks up by email, verifies the password hash matches, returns the user (401 if either check fails)
- Both return the same UserResponse shape — this is why the frontend can use one storeUser() function for both

3.5 Wiring into the app

main.py — relevant lines

```

from app.api.routes import chat, documents, history, feedback, auth
...
app.include_router(auth.router, prefix="/api")

```

- Routes live at /api/signup and /api/login — matches exactly what the frontend's submitAuth() calls

4. Frontend — Session Logic (v2)

Per the v2 changes: session state lives entirely client-side in localStorage under the key insurance-ai-user. No cookies, no server-side session — the backend is stateless and simply trusts whatever user_id the frontend sends with each request.

4.1 On page load

- **loadStoredUser()** reads insurance-ai-user from localStorage into a currentUser variable
- **applyAuthState()** checks if currentUser is set and calls either showAuthView() or showChatView()
- showAuthView() reveals #auth-panel and hides #chat-app (added as a wrapper around the existing chat window + composer, so it toggles as one unit)

4.2 Submitting login or signup

- Both forms POST to a shared helper: **submitAuth(endpoint, payload, submitButton, successMessage)**
- endpoint is '/api/login' or '/api/signup', payload is {email, password} or {name, email, password}
- While the request is in flight, the submit button is disabled (prevents double-submit)
- On success: storeUser(data) saves the returned {id, name, email} to localStorage, then applyAuthState() switches to showChatView()
- On failure: the backend's 401 (login) or 400 (signup — duplicate email) detail message is shown inline via setAuthStatus, no page reload

4.3 Chat requests now carry a real user id

Before (v1) vs after (v2) — the only line that changes in the chat POST

```
// v1
body: JSON.stringify({ user_id: USER_ID, question }) // USER_ID was hardcoded = 1

// v2
body: JSON.stringify({ user_id: currentUser.id, question }) // real logged-in user
```

- Nothing else about the chat request changes — same endpoint, same response handling, same clause-citation and feedback-thumb logic from v1

4.4 Gating: chat and 'New conversation' require login

- The chat-form submit handler checks currentUser first — if it's null, it switches to the login tab, shows 'Please log in...' via setAuthStatus, and returns without calling /api/chat
- The New conversation button has the same guard, so a logged-out user can't even start a fresh session

4.5 Logout

- clearStoredUser() removes insurance-ai-user from localStorage and resets currentUser to null
- applyAuthState() then flips back to showAuthView() — sidebar reverts to guest state

5. Exact Request/Response Contract

Endpoint	Request Body	Success Response	Failure Response
POST /api/signup	{name, email, password}	200: {id, name, email}	400: {detail: 'Email already registered'}
POST /api/login	{email, password}	200: {id, name, email}	401: {detail: 'Invalid email or password'}
POST /api/chat	{user_id, question}	200: {answer, sources}	500 on server error (unrelated to auth)

Because signup and login return the identical shape, the frontend's `storeUser(data)` function works unmodified for both — this is a deliberate symmetry, not a coincidence.

6. Files Touched (Both Sides)

File	Change
app/database/models.py	Added password_hash column to User
app/services/auth_service.py	New — hash_password(), verify_password()
app/schemas/auth.py	New — SignupRequest, LoginRequest, UserResponse
app/api/routes/auth.py	New — POST /signup, POST /login
main.py	Registered auth.router
requirements.txt	Added passlib[bcrypt], bcrypt==4.0.1, email-validator
templates/index.html	Added #auth-panel section, #chat-app wrapper, #user-panel in sidebar
static/style.css	Added .auth-panel/.auth-card/.auth-form/.auth-tab/.user-panel rules, reusing existing design tokens
static/app.js	Added loadStoredUser/storeUser/clearStoredUser, submitAuth(), showAuthView/showChatView/applyAuthState, login gating on chat submit and New conversation

7. Verification Checklist

- Load the app logged out → auth panel shows, chat hidden
- Sign up a new account → lands in chat view, sidebar shows name/email
- Refresh the page → still logged in (localStorage persisted)
- Log out → back to auth panel, sidebar reverts to guest state
- Try sending a message while logged out → blocked with inline message, no request sent to /api/chat
- Two different accounts see two different chat_history rows in MySQL (filtered by user_id)